Project 7

Related Tables, Joins and Views | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 2 hours	students_db_backup.sql	Your GitHub repository URL

The due date is listed on Canvas LMS.

What you are building

In lab7b you built a schema called students_db with one table in it. One table on its own cannot answer the questions a school actually asks. It can tell you who the students are. It cannot tell you what any of them is taking, who is in which class, or which class nobody signed up for.

In this project you add the two tables that make those questions answerable, and you connect them with keys so the server itself refuses to let the data go wrong. Then you write the joins that pull the three tables back together, and you save those joins as views so you never have to type them again.

Table	What it holds	Keys
STUDENT	Already there from lab7b	Primary key on student_id
COURSE	You build it in Step 6	Primary key on course_id
ENROLLMENT	You build it in Step 8	Primary key on both columns, plus two foreign keys

Your starter file is the one you finished last time

This project adds to **students_db_backup.sql**, the file you worked in during lab7b. Do not start a new file. Open your finished copy and keep writing at the bottom of it.

If your copy is missing or you did not get to the end of lab7b, download **students_db_complete.sql** from Canvas, save it into your Project 7 folder, and rename it to **students_db_backup.sql**. It is the finished version of the same file. Then carry on from Step 5.

One rule before you start: label and comment every step

Every step you write has to carry a heading comment with its step number, and every statement has to carry a comment above it saying what it does. It is a graded requirement worth a full point, and without it your instructor cannot tell which statement belongs to which step. Part 2 shows you exactly what that looks like.

Part 1. Get set up again

The lab PCs are wiped every night, so the database you built in lab7b is gone. The file is not gone, because the file was on OneDrive. That is the whole point of working this way. These five steps put the database back.

Step 1. Make a folder on OneDrive for this project

Sign in to OneDrive using the OneDrive app on the PC, not the browser. A folder you make in the browser does not always appear on the PC straight away, and VS Code can only open what the PC can see. Make the folder through the app and it is there instantly.

In File Explorer, open your OneDrive folder and create:

```
CIS4720_Project7
```

Check that the folder shows a small cloud or a green tick next to it. That tick is OneDrive telling you the folder is backed up. No tick means the folder is sitting on a PC that gets wiped tonight.

Step 2. Put your starter file in that folder

Copy your finished **students_db_backup.sql** from your lab7b folder into CIS4720_Project7. Copy it, do not move it, so your lab7b submission stays as you left it. If you are using students_db_complete.sql instead, save it here and rename it to students_db_backup.sql now.

Step 3. Create the database at the command prompt

Do this before you open VS Code. VS Code can connect to a schema that exists, but it will not create one for you, and the schema you made in lab7b went away with the nightly wipe. Open the Windows command prompt and start the MySQL client:

```
mysql -u root -p
```

Type the password your instructor gave you. Nothing appears as you type, which is normal. When you see the **mysql>** prompt, create the schema and leave:

```
CREATE DATABASE students_db;
SHOW DATABASES;
exit
```

> **Skip this step and nothing in Part 2 will work**
>
> If you go straight to VS Code, the connection fails with **ERROR 1049 (42000): Unknown database 'students_db'**. That error is not a problem with VS Code or with your file. It means the schema is not there yet, and only this step creates it.

Step 4. Open the folder in VS Code and connect

In VS Code choose **File**, then **Open Folder**, and pick CIS4720_Project7. Open the folder, not the file. SQLTools saves your connection inside the folder you have open, so if you open only the file there is nowhere for it to save and you will set the connection up again every time.

You need both of these extensions. You installed them in lab7b, so they are probably still there. Check anyway:

Extension	Publisher
SQLTools	mtxr
SQLTools MySQL/MariaDB/TiDB	mtxr

Then click the SQLTools icon in the left bar, choose **Add New Connection**, pick **MySQL**, and fill in:

Field	What to enter
Connection name	students_db
Server	localhost
Port	3306

Field	What to enter
Database	students_db
Username	root
Password	The one your instructor gave you

Click **Test Connection**, then **Save Connection**. A green tick means you are in.

Step 5. Run your starter file once, top to bottom

Open students_db_backup.sql. Right click anywhere in it and choose **Run This File**. Every CREATE and INSERT in the file runs, and in a couple of seconds your database is back exactly as you left it in lab7b.

Confirm it worked before you write anything new:

```
SELECT COUNT(*) AS students, ROUND(AVG(gpa), 2) AS average_gpa
FROM STUDENT;
```

You should get **5** and **3.43**. If you get an error instead, stop here and sort it out. Nothing after this point works without those five students.

> **When the first run is done, comment out the create and insert lines**
>
> Run the file a second time and every CREATE and every INSERT fails, because the tables already exist and the rows are already there. The errors are harmless but they bury the results you actually want to read.
>
> So once the first run has worked, comment out Sections 2, 3 and 4 and the lab7b steps. Leave the SELECT statements alone. SELECT only reads, so it is safe to run as many times as you like.
>
> To comment one line, put two dashes and a space at the front of it. To comment a whole block, select the lines and press **Ctrl + /** in VS Code, which puts two dashes in front of each line you selected.

> **Do not use /* and */ to comment out a block in this file**
>
> SQL block comments do not nest. Your file already has comments in it, and the first ***/** inside the region you wrapped closes your block early. The rest of the region runs anyway, and the leftover ***/** gives you **error 1064**, a syntax error pointing at a line that looks perfectly fine.
>
> Two dashes per line never has this problem. Use **Ctrl + /** and move on.

Part 2. Add two related tables

Everything you write from here goes at the bottom of students_db_backup.sql, in step order.

> **Label every step. This is a requirement, not a suggestion**
>
> Before the statements for each step, put a heading comment with the step number on it. Above each individual statement, put a short comment saying what that statement does, in your own words.
>
> This is graded, and it is worth a full point on its own. The reason is simple. What your instructor opens is one long .sql file with roughly forty statements in it. Without the labels there is no way to tell which statement was meant to be Step 10 and which was meant to be Step 11, so work you actually did correctly cannot be given credit for the step it belongs to. Labeling takes you about a minute per step and it is the difference between a graded project and a guessed one.

What a labeled step looks like

Use this shape for all eight of the steps in Parts 2, 3 and 4:

```
-- ====================================================================
-- PROJECT 7 STEP 6. Create the COURSE table
-- ====================================================================

-- create the table that holds the classes on offer. course_id is the
-- primary key, so two classes can never share an id
CREATE TABLE COURSE (
    course_id    INT          NOT NULL,
    course_code  VARCHAR(10)  NOT NULL,
    PRIMARY KEY (course_id)
);

-- show the structure so I can confirm course_id came out as PRI
DESCRIBE COURSE;
```

Three things make that work, and all three are graded:

The heading names the step number, so your instructor can find it.

Every statement has a comment above it saying what it does.

The comments are in your own words, not copied out of this handout. A comment that just repeats the SQL back, such as *creates a table called COURSE* above a CREATE TABLE COURSE statement, does not earn the point. Say why the statement is there or what it is for.

Here is the same step with no labels. Everything runs, and it is worth less:

```
CREATE TABLE COURSE (
    course_id    INT          NOT NULL,
    course_code  VARCHAR(10)  NOT NULL,
    PRIMARY KEY (course_id)
);
DESCRIBE COURSE;
```

Nothing is wrong with that SQL. The problem is that it arrives in the middle of forty other statements with nothing to say which step it answers, so it has to be worked out rather than read.

The eight headings you need, exactly as they appear in this handout:

Heading to put in your file	Covers
-- PROJECT 7 STEP 6. Create the COURSE table	Step 6
-- PROJECT 7 STEP 7. Add rows to COURSE	Step 7
-- PROJECT 7 STEP 8. Create the ENROLLMENT table	Step 8
-- PROJECT 7 STEP 9. Add rows to ENROLLMENT	Step 9
-- PROJECT 7 STEP 10. Join one, the three table join	Step 10
-- PROJECT 7 STEP 11. Join two, the outer join	Step 11
-- PROJECT 7 STEP 12. View one, STUDENT_SCHEDULE	Step 12
-- PROJECT 7 STEP 13. View two, COURSE_HEADCOUNT	Step 13

If you are using students_db_complete.sql, those eight headings are already in the file near the bottom, waiting for you. Write your statements underneath each one. If you are using your own file from lab7b, type the headings in yourself as you go.

> **Build them in this order and do not swap it round**
>
> COURSE has to exist before ENROLLMENT, because ENROLLMENT points at it. Create ENROLLMENT first and you get **ERROR 1005 (HY000): Can't create table, errno: 150, Foreign key constraint is incorrectly formed**. That message does not mention the missing table, which is why it confuses people. It means you pointed at something that is not there yet.
>
> The same rule applies to the rows. A course has to exist before anyone can enroll in it.

Step 6. Create the COURSE table

This one stands on its own. It has a primary key and nothing else points out of it.

```
-- the classes on offer. course_id is the primary key, so no two
-- courses can share an id and the id can never be left empty
CREATE TABLE COURSE (
    course_id    INT          NOT NULL,
    course_code  VARCHAR(10)  NOT NULL,
    course_title VARCHAR(50)  NOT NULL,
    credits      INT,
    dept         VARCHAR(30),
    PRIMARY KEY (course_id)
);

DESCRIBE COURSE;
```

In the DESCRIBE output, course_id shows **PRI** in the Key column. That is the primary key.

Step 7. Add rows to COURSE

Five classes. One of them has nobody in it on purpose, and you will see why in Step 11.

```
-- five classes across three departments
INSERT INTO COURSE VALUES (101,'CIS4720','Database Management and SQL',3,'CIS');
INSERT INTO COURSE VALUES (102,'CIS2000','Intro to Programming',3,'CIS');
INSERT INTO COURSE VALUES (103,'MATH1600','Calculus I',4,'Mathematics');
INSERT INTO COURSE VALUES (104,'BUS2100','Principles of Management',3,'Business');
INSERT INTO COURSE VALUES (105,'CIS4850','Advanced Python',3,'CIS');

SELECT * FROM COURSE;
```

Five rows. Check them on screen before you move on.

Step 8. Create the ENROLLMENT table

A student can take many classes, and a class can hold many students. Neither table has room for that on its own, so you need a third table in the middle to hold the pairings. Each row is one student in one class.

```
-- one row per student per class. the primary key is both columns
-- together, so the same student cannot be enrolled in the same
-- class twice. the two foreign keys make the server check that
-- the student and the class both really exist
CREATE TABLE ENROLLMENT (
    student_id INT         NOT NULL,
    course_id  INT         NOT NULL,
    term       VARCHAR(15),
    grade      CHAR(1),
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES STUDENT (student_id),
    FOREIGN KEY (course_id)  REFERENCES COURSE  (course_id)
);

DESCRIBE ENROLLMENT;
```

What those two kinds of key are doing

The **primary key** is **(student_id, course_id)**, both columns at once. That is a composite key. One column on its own would be no use here, because student 1 appears on several rows and course 101 appears on several rows. The pair is what has to be unique.

A **foreign key** is a column that has to match a primary key in another table. student_id must match a student_id that already exists in STUDENT, and course_id must match a course_id that already exists in COURSE. The server enforces this on every insert, update and delete, which means your data cannot drift into listing an enrollment for a student who does not exist.

Step 9. Add rows to ENROLLMENT

Eight enrollments. Notice that student 5 is not in the list, which matters in Step 11.

```
-- eight pairings of a student with a class
INSERT INTO ENROLLMENT VALUES (1,101,'Fall 2026','A');
INSERT INTO ENROLLMENT VALUES (1,102,'Fall 2026','B');
INSERT INTO ENROLLMENT VALUES (1,103,'Spring 2026','A');
INSERT INTO ENROLLMENT VALUES (2,101,'Fall 2026','B');
INSERT INTO ENROLLMENT VALUES (2,104,'Fall 2026','C');
INSERT INTO ENROLLMENT VALUES (3,101,'Fall 2026','A');
INSERT INTO ENROLLMENT VALUES (3,103,'Spring 2026','A');
INSERT INTO ENROLLMENT VALUES (4,102,'Fall 2026','C');

SELECT COUNT(*) AS enrollment_rows FROM ENROLLMENT;
```

You should get **8**.

Now prove the keys are doing their job. Run these three one at a time. All three are **supposed** to fail, so read each message and work out which key stopped it. Put a comment above each one saying what you expect, then comment the three statements out when you are done so they do not break your next run.

```
-- a student who does not exist
INSERT INTO ENROLLMENT VALUES (99,101,'Fall 2026','A');

-- a class that does not exist
INSERT INTO ENROLLMENT VALUES (1,999,'Fall 2026','A');

-- a pairing that is already in the table
INSERT INTO ENROLLMENT VALUES (1,101,'Spring 2027','B');
```

The first two give **ERROR 1452**, a foreign key constraint fails. The third gives **ERROR 1062**, duplicate entry '1-101' for key 'PRIMARY'. Three bad rows that never reached the table, with no code of yours involved.

Part 3. Write two joins

You now have the data spread across three tables, which is correct but hard to read. A join puts it back together for one query, matching rows from each table using the key columns they share.

Step 10. Join one, all three tables together

This answers who is taking what. ENROLLMENT is the table in the middle, so you join STUDENT to it and then join COURSE to it as well. Each ON says which column matches which.

```
-- every enrollment written out in full, with the student's name
-- and the class title instead of the id numbers
SELECT s.first_name, s.last_name, c.course_code, c.course_title,
       e.term, e.grade
FROM STUDENT s
JOIN ENROLLMENT e ON s.student_id = e.student_id
```

```
JOIN COURSE     c ON e.course_id  = c.course_id
ORDER BY s.last_name, c.course_code;
```

Eight rows, the same eight pairings you inserted, with readable names in place of ids. The letters s, e and c after the table names are aliases. They save you typing STUDENT.first_name every time, and once a query touches three tables they are the only thing that keeps it readable.

> **Nobody with no classes shows up here**
>
> Elena Vargas is student 5 and she is in your STUDENT table, but she is not in those eight rows. A plain JOIN only returns rows that match on both sides, and she has no enrollment to match. Step 11 is how you find her.

Step 11. Join two, an outer join that keeps the rows with no match

LEFT JOIN keeps every row from the table on the left whether or not it has a match on the right. Where there is no match you get NULL in the right hand columns instead of losing the row.

```
-- every student and how many classes they are taking, including
-- the ones taking none
SELECT s.first_name, s.last_name, COUNT(e.course_id) AS courses_taken
FROM STUDENT s
LEFT JOIN ENROLLMENT e ON s.student_id = e.student_id
GROUP BY s.student_id, s.first_name, s.last_name
ORDER BY courses_taken DESC, s.last_name;
```

All five students come back this time. Elena shows **0**:

first_name	last_name	courses_taken
Ana	Reyes	3
Marcus	Bell	2
Priya	Nair	2
Devon	Rao	1
Elena	Vargas	0

Swap JOIN for LEFT JOIN in Step 10 and run it again, and Elena appears with NULL in every course column. That is the difference between the two joins, in one row.

The same idea finds the classes nobody signed up for. Run this one too:

```
-- classes with no students in them
SELECT c.course_code, c.course_title
FROM COURSE c
LEFT JOIN ENROLLMENT e ON c.course_id = e.course_id
WHERE e.student_id IS NULL;
```

One row, CIS4850 Advanced Python. A report that only used a plain JOIN would say that class does not exist, which is the kind of quiet mistake an outer join is there to prevent.

Part 4. Save your work as two views

A view is a stored SELECT statement you can query as if it were a table. Both of your joins are worth keeping, so turn them into views. After this you can get the same answer with SELECT * and no join at all.

Step 12. View one, STUDENT_SCHEDULE

This wraps up the three table join from Step 10.

```
-- the three table join saved under a name, so the full class
-- schedule can be read without writing the join again
CREATE VIEW STUDENT_SCHEDULE AS
SELECT s.first_name, s.last_name, c.course_code, c.course_title,
       e.term, e.grade
FROM STUDENT s
JOIN ENROLLMENT e ON s.student_id = e.student_id
JOIN COURSE     c ON e.course_id  = c.course_id;
```

Now use it. No join in sight:

```
SELECT * FROM STUDENT_SCHEDULE;

SELECT * FROM STUDENT_SCHEDULE WHERE last_name = 'Reyes';

SELECT * FROM STUDENT_SCHEDULE WHERE course_code = 'CIS4720';
```

Eight rows, then three rows for Ana, then three rows for CIS4720. The view is where the join lives now, and you can filter it like any table.

Step 13. View two, COURSE_HEADCOUNT

This one counts. It is built on the outer join so that an empty class still appears, with a count of zero rather than being dropped.

```
-- how many students are in each class, counting the empty
-- classes as zero instead of leaving them out
CREATE VIEW COURSE_HEADCOUNT AS
SELECT c.course_code, c.course_title,
       COUNT(e.student_id) AS students_enrolled
FROM COURSE c
LEFT JOIN ENROLLMENT e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_code, c.course_title;

SELECT * FROM COURSE_HEADCOUNT ORDER BY students_enrolled DESC;
```

Five rows, busiest class first, with Advanced Python on zero:

course_code	course_title	students_enrolled
CIS4720	Database Management and SQL	3
CIS2000	Intro to Programming	2

course_code	course_title	students_enrolled
MATH1600	Calculus I	2
BUS2100	Principles of Management	1
CIS4850	Advanced Python	0

Last, confirm everything is there and that the server knows which items are tables and which are views:

```
SHOW FULL TABLES;
```

Nine rows, because your lab7b work is still in the schema alongside the new work:

Tables_in_students_db	Table_type	Came from
COURSE	BASE TABLE	Step 6
COURSE_HEADCOUNT	VIEW	Step 13
ENROLLMENT	BASE TABLE	Step 8
GPA_SUMMARY	VIEW	lab7b
HIGH_GPA	VIEW	lab7b
STUDENT	BASE TABLE	lab7b
STUDENT_COPY	BASE TABLE	lab7b
STUDENT_GPA	VIEW	lab7b
STUDENT_SCHEDULE	VIEW	Step 12

The four rows marked Step 6 through Step 13 are the ones this project is about. If you did not finish lab7b you will see fewer of the lab7b rows, and that is fine. What matters is that COURSE and ENROLLMENT show as BASE TABLE, and STUDENT_SCHEDULE and COURSE_HEADCOUNT show as VIEW.

> **One of these two views cannot be written through**
>
> In lab7b you saw that a view over a single table accepts an UPDATE and passes it down to the real table, while a view with an aggregate in it refuses. Try it on COURSE_HEADCOUNT:
>
> `UPDATE COURSE_HEADCOUNT SET students_enrolled = 99 WHERE course_code = 'CIS4720';`
>
> It is refused with **error 1288, the target table COURSE_HEADCOUNT of the UPDATE is not updatable**. There is no single row behind that 3 to change. It is a count over three rows, and the server has no way to work backwards from 99 to which enrollments you meant. Comment this statement out after you have seen the error.

Part 5. Publish to a new GitHub repository

Your file is on OneDrive, which keeps it safe but does not show anyone what you did. GitHub does. You made a repository for lab7b, and this project needs a separate one of its own, so each piece of work has its own history.

Step 14. Set up VS Code for GitHub

If you already published lab7b from this PC, you are probably still signed in, so check quickly and then skip to Step 15. If the PC was wiped or you are on a different machine, do all of this.

First, Git itself has to be installed. Open the command prompt and run:

```
git --version
```

A version number means you are fine. **'git' is not recognized** means it is not installed, and nothing in this part will work until it is. Tell your instructor.

Then sign VS Code in to your GitHub account:

1. In VS Code, click the **Accounts** icon at the bottom of the left bar.
2. Choose **Sign in with GitHub**. Your browser opens.
3. Sign in to github.com if you are not already, then click **Authorize**.
4. The browser asks to hand you back to VS Code. Allow it.
5. Back in VS Code, the Accounts icon now shows your GitHub username. That is how you know it worked.

You should also have the **GitHub Pull Requests and Issues** extension installed, publisher GitHub. Search the Extensions panel for it if it is not there.

> **Sign out before you leave the lab**
>
> This is a shared PC. Your GitHub sign in stays on it until somebody removes it, and anyone who sits down afterwards is signed in as you. When you are finished, click **Accounts**, then your username, then **Sign Out**.
>
> The same goes for your SQLTools connection, which stores the MySQL password in plain text in **.vscode/settings.json** inside your project folder. Never let that folder reach GitHub with a real password in it. The password in this lab is a throwaway one on a lab PC, which is the only reason it is safe here.

Step 15. Publish the folder as a new repository

Make sure CIS4720_Project7 is the folder open in VS Code, and that your saved students_db_backup.sql is in it.

1. Click the **Source Control** icon in the left bar. It looks like a small branching line.
2. Click **Publish to GitHub**.
3. VS Code asks for a repository name. Type **cis4720-project7**. This has to be a new name, not the lab7b one.
4. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
5. VS Code asks which files to include. Tick **students_db_backup.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.
6. Wait for the notification saying it published. It takes a few seconds.

> **If it says the repository already exists**
>
> You cannot have two repositories with the same name on one account. If cis4720-project7 is taken because of an earlier attempt, either delete the old one on github.com under Settings, or publish under **cis4720-project7b** and submit that URL instead.

Step 16. The website and the extension are two ways to the same place

You can create a repository from the GitHub website or from inside VS Code, and students regularly get these two mixed up and end up with an empty repository on the site and their work still only on the PC. Here is the difference.

	From the website	From VS Code
Where you start	github.com in a browser	The Source Control panel
What you get	An empty repository with no files in it	A repository with your files already in it
Next thing you do	Clone it, copy files in, then commit and push	Nothing. It is already done
Good for	Starting something from scratch	Work that already exists in a folder, like this project

You already have a folder with a finished file in it, so **Publish to GitHub** from VS Code is the right route and the website route would be extra work. The **GitHub Pull Requests and Issues** extension is a third thing again. It does not create repositories. It lets you read and review other people's proposed changes from inside VS Code, which you will use later in the course and do not need today.

Step 17. Check on github.com that your work actually arrived

Do not skip this. A repository that published without the file still looks fine from inside VS Code.

1. Go to github.com and open your profile, then **Repositories**.
2. Click **cis4720-project7**.
3. You should see **students_db_backup.sql** listed. Click it and read it on the page.
4. Check your Part 2, 3 and 4 statements are all there, with your comments above them. What you see on this page is exactly what gets graded.
5. Copy the URL from the address bar. It looks like **github.com/yourusername/cis4720-project7**.

> **Test the URL the way your instructor will open it**
>
> Open a private or incognito browser window and paste the URL in. If you can read the file without signing in, so can anyone. If you get a **404**, the repository is private. Go to the repository, then **Settings**, scroll to the bottom and change the visibility to public.
>
> A 404 at grading time is a zero, and it is the single most common way to lose all ten points on a project that was otherwise finished.

Step 18. Submit the URL on Canvas

Paste the repository URL into the Project 7 submission box on Canvas. The URL is what gets you credit, so submit it even if you did not get all the way through the SQL. Partial work that your instructor can read scores better than finished work nobody can reach.

What to turn in

The URL of your **cis4720-project7** repository, pasted into Canvas. That repository has to contain:

A file called **students_db_backup.sql** holding everything from lab7b plus all of your Part 2, 3 and 4 work.

A numbered heading on each of Steps 6 through 13, and a comment in your own words above every statement. This is required.

No **.vscode** folder, because that one holds your password.

How it is graded

Steps	What is being checked	Points
1 to 5	Folder on OneDrive and the starter file running again	1.0
6 to 7	COURSE created with a primary key, and five rows in it	1.5
8 to 9	ENROLLMENT created with a composite primary key and two foreign keys, and eight rows in it	2.0
10 to 11	One join across all three tables, and one outer join	2.0
12 to 13	STUDENT_SCHEDULE and COURSE_HEADCOUNT both created and queried	2.0
All	Required. A numbered heading on every step, and a comment in your own words above every statement	1.0
14 to 18	A new public repository with the file in it, URL submitted	0.5

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1049, unknown database 'students_db'	The schema is not there. Go back and do Step 3 at the command prompt.
ERROR 1005, errno 150, foreign key incorrectly formed	You created ENROLLMENT before COURSE. Create COURSE first, then ENROLLMENT.
ERROR 1452, a foreign key constraint fails	The student or the class in that row does not exist yet. Check the id, and check that Step 7 ran.
ERROR 1062, duplicate entry for key 'PRIMARY'	That exact row is already in the table. You probably ran the inserts twice.
ERROR 1050, table already exists	A second run. Comment out the CREATE statements and run it again.
ERROR 1288, the target table is not updatable	You are trying to UPDATE a view with an aggregate in it. Expected in Step 13.
ERROR 1451, cannot delete a parent row	Something in ENROLLMENT still points at the row you are deleting. Delete the enrollment rows first.
ERROR 1064, a syntax error on a line that looks fine	Usually a block comment. Replace /* and */ with two dashes per line.

Starting the whole thing over

If your tables end up in a state you cannot untangle, drop them and run your file again. The order matters: drop the child before the parent, or the foreign key refuses with **error 1451**.

```
DROP TABLE ENROLLMENT;
DROP TABLE COURSE;
```

ENROLLMENT points at COURSE, so ENROLLMENT has to go first. Dropping in the reverse of the order you built things in is the general rule, and it is the same rule as building the parent first, seen from the other end.